

DS	Session	Stack	Question Information	Level 1	Challenge 42
----	---------	-------	----------------------	---------	--------------

Question description

Rajinikanth organized a technical round interview in an Animation company for the set of computer science candidates.

the task is to implement stack operations for two stacks and merge the stacks into one.

Get two sets of stack elements and reverse them, then merge them into one stack.

Rajinikanth has given the deadline of only 15 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit?

Function Description

- a) push(): Adds the new item at the beginning of the linked list using the first pointer.
- b) pop(): Removes an item from the beginning using the first pointer.
- c) merge(): Links the first pointer second stack as next of the last pointer of the first list.

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations that need to be performed

The next line indicates the n number stack elements

Output Format:

First line indicates the top of the element of the stack

the second line indicates the top of the element after the pop operation

Logical Test Cases

Test Case 1

INPUT (STDIN)

5 8
9 8 7 6 5
5 6 7 8 3 11 2 3

EXPECTED OUTPUT

5 6 7 8 9 3 2 11 3 8 7 6 5

Test Case 2

INPUT (STDIN)

10 4
91 18 17 16 15 15 16 17 18 13
11 12 13 14

EXPECTED OUTPUT

13 18 17 16 15 15 16 17 18 91 14 13 12 11

Mandatory Test Cases

Test Case 1

KEYWORD

void push(int data, mystack* ms)

Test Case 2

KEYWORD

int pop(mystack* ms)

Test Case 3

KEYWORD

void merge(mystack* ms1, mystack* ms2)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

435

Test Case 3

NLOC

90

```
Stack2.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int data;
6     struct Node* next;
7 } Node;
8
9 typedef struct mystack {
10     Node* top;
11 } mystack;
12
13 mystack* createStack() {
14     mystack* ms = (mystack*)malloc(sizeof(mystack));
15     ms->top = NULL;
16     return ms;
17 }
18
19 void push(int data, mystack* ms) {
20     Node* temp = (Node*)malloc(sizeof(Node));
21     temp->data = data;
22     temp->next = ms->top;
23     ms->top = temp;
24 }
25
```

```

26 int pop(mystack* ms) {
27     if (ms->top == NULL) return -1;
28     Node* temp = ms->top;
29     int val = temp->data;
30     ms->top = ms->top->next;
31     free(temp);
32     return val;
33 }
34
35 void merge(mystack* ms1, mystack* ms2) {
36     if (ms1->top == NULL) {
37         ms1->top = ms2->top;
38         return;
39     }
40     Node* curr = ms1->top;
41     while (curr->next != NULL) {
42         curr = curr->next;

```

main | CH.SC.U4CSE25225 | Stack2.c

Ln 75, Col 1 Spaces: 4 UTF-8 CRLF C

```
43     }
44     curr->next = ms2->top;
45 }
46
47 int main() {
48     int n, m;
49     if (scanf("%d %d", &n, &m) != 2) return 0;
50
51     mystack* ms1 = createStack();
52     mystack* ms2 = createStack();
53
54     for (int i = 0; i < n; i++) {
55         int val;
56         scanf("%d", &val);
57         push(val, ms1);
58     }
59     for (int i = 0; i < m; i++) {
```

```
60     int val;
61     scanf("%d", &val);
62     push(val, ms2);
63 }
64
65 merge(ms1, ms2);
66
67 Node* curr = ms1->top;
68 while (curr != NULL) {
69     printf("%d ", curr->data);
70     curr = curr->next;
71 }
72 printf("\n");
73 return 0;
74}
75
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

5 8

9 8 7 6 5

5 6 7 8 3 11 2 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack2.exe

5 6 7 8 9 3 2 11 3 8 7 6 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

10 4

91 18 17 16 15 15 16 17 18 13

11 12 13 14

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Stack2.exe

13 18 17 16 15 15 16 17 18 91 14 13 12 11

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>